Kunskapskontroll 3: Tjugoett

Utbildning	Mjukvaruutvecklare med AI-inriktning – MAI26 (Malmö / Halmstad)
Kurs	Programmering och objektorienterad utveckling i C#
Kursansvarig	Thomas Frank
Betyg	IG / G / VG
Delas ut	Måndag 12/10
Redovisning	Onsdag 21/10 (Malmö), fredag 23/10 (Halmstad)
Inlämning	Lämnas in i kursen på Learnpoint senast fredag 23/10 kl 23:59
Genomförs	I grupp om 4–5 personer, med en individuell del var

Syfte

Den här uppgiften prövar om ni kan bygga ett program **tillsammans**. Inte fem program bredvid varandra, utan ett, där era delar är beroende av varandra och där ni måste komma överens om hur de ska passa ihop.

Det mesta av arbetet är gemensamt. Ovanpå det äger var och en ett eget tillval, på en egen branch, som bedöms individuellt.

Ni arbetar i ett riktigt Git-flöde med skyddade brancher, pull requests och kodgranskning — samma arbetssätt som ni kommer att möta i yrkeslivet.

Vad ni ska bygga

Ett spelbart tjugoett i konsolen, för en till fem spelare mot en given.

Programmet ska gå att spela igenom från start till slut utan att krascha, och spelarnas resultat ska sparas till fil så att de finns kvar nästa gång programmet startas.

Reglerna står nedan, fullständigt. Det är de reglerna som gäller — inte den variant ni spelat någon annanstans.

Ordlista

Klasser, metoder och variabler skrivs på **engelska**. Allt användaren ser skrivs på **svenska**. Här är orden ni kommer att behöva.

Engelska	Svenska	Noteringar
suit	färg	Hjärter, ruter, klöver, spader
rank	valör	Två till ess
deck	kortlek	
to shuffle	blanda	
to deal	ge, dela ut	
hand	hand	
dealer	givaren	
hole card	hålkortet	Givarens dolda kort
upcard	öppet kort	Givarens synliga kort
face card	klätt kort	Knekt, dam, kung
hit	köpa	
stand	stanna	
bust	spräcka	
push	lika	
soft / hard hand	mjuk / hård hand	
blackjack, natural	naturlig tjugoett	
chips	marker	Tillval 4
bet	insats	Tillval 4
payout	utbetalning	Tillval 4
double down	dubbla	Överkurs
split	dela par	Överkurs
insurance	försäkring	Tillval 4, om tid finns
shoe	sko	Tillval 5
cut card	skiljekort	Tillval 5

Spelets regler

Mål

Få en hand vars värde kommer så nära 21 som möjligt utan att överstiga 21, och som är högre än givarens hand.

Kortens värde

Kort	Värde
2–10	Sitt eget värde
Knekt, dam, kung	10
Ess	1 eller 11 — det som är bäst för handen utan att spräcka

En hand där ett ess räknas som 11 kallas **mjuk**. ♥A + ♠6 är en mjuk 17. Drar man ♦9 till den blir handen A + 6 + 9, och nu måste esset räknas som 1: en **hård** 16. Esset byter alltså värde under handens gång. Det är den detalj som ställer till det i koden om man räknar värdet en gång och sparar det.

Given

1. Varje spelare får två kort, öppet.
2. Givaren får två kort: ett öppet och ett dolt. Det dolda kallas hålkortet.
3. Har en spelare 21 på sina två första kort — ess plus ett tiokort — har hen **naturlig tjugoett**.

Spelarnas turer

Spelarna spelar en i taget, i tur och ordning. Varje spelare väljer upprepade gånger mellan:

- **Köpa** – ta ett kort till
- **Stanna** – avsluta sin tur

Går handen över 21 har spelaren **spräckt**. Handen är förlorad omedelbart, oavsett vad givaren sedan gör.

Givarens tur

Givaren spelar efter att alla spelare är klara, och följer en fast regel — givaren väljer aldrig:

1. Vänd hålkortet.
2. Köp så länge handens värde är **16 eller lägre**.
3. Stanna på **17 eller högre**, även om det är en mjuk 17.

Givaren tittar alltså **inte** på hålkortet i förväg, inte ens när det öppna kortet är ett ess eller ett tiokort. Rundan spelas alltid färdigt.

NBI/HANDELSAKADEMIN
GÖTEBORG · MALMÖ · HALMSTAD · VÄXJÖ · LULEÅ
+46 (0) 31-794 67 00
www.nbi-handelsakademin.se

Avgörande
Varje spelare jämförs med givaren för sig:

Situation	Utfall
Spelaren har spräckt	Förlust, även om givaren också spräcker
Givaren har spräckt, spelaren inte	Vinst
Ingen har spräckt	Högst hand vinner
Lika värde	Lika — rundan räknas varken som vinst eller förlust
Naturlig tjugoett mot en hand på 21 av tre kort eller fler	Den naturliga vinner
Naturlig tjugoett på båda sidor	Lika

Kortleken
Grundversionen använder **en** kortlek om 52 kort. När färre än 15 kort är kvar tas en ny, blandad lek fram inför nästa runda.

Blandningen ska vara riktig. Använd Fisher–Yates: gå baklänges genom leken och byt plats på varje kort med ett slumpmässigt valt kort bland dem som ligger före. Skapa **en** `Random` som delas av hela programmet — skapar ni en ny varje gång ni blandar får ni samma lek om och om igen, eftersom den såddas från klockan.

Vad grundversionen inte innehåller
Inga marker och inga insatser — programmet räknar bara vunna, förlorade och lika rundor per spelare. Inga satsningar, ingen försäkring, inget dubblande och ingen delning av par. Flera av de sakerna finns som tillval.

Förslag på klasser

Det här är **förslag**, inte krav — men börjar ni härifrån slipper ni en dag av diskussion.

Klass	Ansvar
`Suit`, `Rank`	Enum:ar för färg och valör
`Card`	Ett kort: färg och valör
`Deck`	Bygger leken, blandar, delar ut, vet hur många kort som är kvar
`Hand`	En samling kort som kan räkna ut sitt eget värde och veta om den är mjuk, spräckt eller tjugoett
`Participant`	Basklass: har namn och hand, kan ta emot kort
`Player`	Ärver `Participant`. Fattar ett beslut varje tur
`Dealer`	Ärver `Participant`. Har hålkortet och den fasta regeln i stället för ett beslut
`Round`	En giv: dela ut, spelarnas turer, givarens tur, avgörande
`Game`	Meny och spelloop
`Move`	Enum: `Hit`, `Stand`
`Outcome`	Enum: `Win`, `Lose`, `Push`, `Blackjack`
`GameStorage`	Läser och skriver spelare och resultat till fil
`ConsoleUi`	All in- och utmatning. Visar händer, frågar, validerar svar

Fyra designfrågor ni behöver svara på
Diskutera dem i gruppen innan ni börjar koda, och skriv ner svaren — ni behöver dem i era rapporter.

Ska `Card` ha en `Value`? Fundera på esset innan ni svarar.

Är givaren en spelare? Båda har en hand och kan spräcka. Men givaren har inget val och följer en regel. Vad delar de, och vad gör de inte?

Vem bestämmer vad en spelare gör? Om `Player` frågar `Console` direkt kan ni aldrig lägga till en datorspelare. Var ska beslutet komma ifrån?

Var ligger givarens regel? På `Dealer`, eller i `Round`? Vad händer med `Round` om den ska känna till allas regler?

En regel om struktur
Ingen `Console` i regelklasserna. `Card`, `Deck`, `Hand`, `Participant`, `Player`, `Dealer` och `Round` får varken skriva ut eller läsa in något. All kontakt med användaren sker i `ConsoleUi`.

Det är inte en formalitet. Det är det enda som gör att tillval 1 och 2 går att bygga utan att riva upp grunden — och ni kommer att märka det.

Krav på den gemensamma delen

Innan någon börjar med sitt tillval ska det här fungera:

[ ] En till fem spelare kan läggas till och spela

[ ] Korten delas ut enligt reglerna och givaren följer sin regel

[ ] Mjuka och hårda händer räknas rätt, inklusive flera ess i samma hand

[ ] Naturlig tjugoett hanteras enligt tabellen ovan

[ ] Felaktig inmatning hanteras utan att programmet kraschar

[ ] Spelare och resultat sparas till fil och läses tillbaka vid nästa start

[ ] Programmet går att spela många rundor i följd utan att kortleken tar slut

Sikta på att ha det här klart i slutet av den första veckan.

De fem tillvalen

Var och en i gruppen väljer **ett** tillval. Två personer i samma grupp får inte välja samma. Meddela er lärare vilket ni valt senast onsdag den 14 oktober.

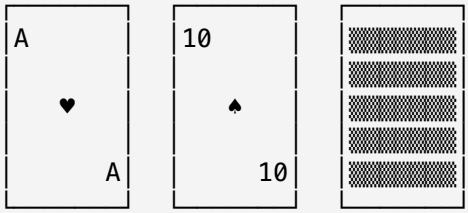
Lös det som ni vill, men **minst en egen klass** ska ingå — något som är ditt, som har ett namn och ett tydligt ansvar, och som du kan peka på vid redovisningen.

Tre av tillvalen kräver ändringar i den gemensamma koden. Det är meningen. De ändringarna görs som pull requests som granskas av den som skrev koden från början, och de bör vänta tills grunden står — alltså vecka två.

1. Kortgrafik i konsolen
Rör bara ny kod. Kan påbörjas tidigt.

Grundversionen visar kort som text: `♥A ♠10`. Ditt tillval ritar dem som kort.

Ett kort blir en ruta med ram, valören i två hörn och färgen i mitten. Givarens hålkort ritas med baksidan upp. Korten i en hand ska stå **bredvid** varandra, inte under.



Det är sidledsplaceringen som är den tekniska knuten. Du kan inte skriva ut ett kort i taget — du måste bygga rad ett för alla kort, sedan rad två, och så vidare. Låt din renderare returnera raderna som `string[]` i stället för att skriva ut dem, så kan `ConsoleUi` fläta ihop dem.

Hjärter och ruter ska vara röda, klöver och spader svarta. Håll `Console.ForegroundColor` i `ConsoleUi` — din renderare ska bara svara för formen.

För klädda kort finns schackglyferna ♚ ♛ ♞ i Unicode. De duger bättre än försök att teckna ansikten.

Det här tillvalet är ett test av grundkoden. Om ingen regelklass behöver ändras när presentationen byts ut, då var ansvarsfördelningen rätt. Måste du ändra i `Hand` eller `Round` för att byta utseende, så har ni hittat en läcka. Skriv om det i din rapport — båda utfallen är intressanta.

2. Datorspelare med strategi

Rör bara ny kod, förutsatt att gruppen löst designfråga tre rätt. Kan påbörjas tidigt.

Lägg till datorstyrda spelare vid bordet. De ska kunna sitta samtidigt som mänskliga spelare, och flera datorspelare ska kunna ha **olika** strategier samtidigt.

Bygg ett gemensamt gränssnitt — till exempel `IPlayerStrategy` med en metod som tar emot spelarens hand och givarens öppna kort och returnerar ett `Move`. Implementera minst två:

- En enkel: köp så länge handen är under 17.
- En som följer grundstrategin: slå upp rätt drag i en tabell utifrån den egna handens värde, om den är mjuk eller hård, och vilket kort givaren visar.

Grundstrategitabellen finns publicerad och du får använda en färdig tabell — men du ska kunna förklara hur din kod läser den.

Det här är polymorfism som gör något begripligt: samma anrop, olika beteende, utbytbart under körning.

3. Husregler från konfigurationsfil

Kräver ändringar i gemensam kod — `Dealer`, `Round` och `Deck`. Vänta tills grunden står.

Riktiga kasinon spelar inte med samma regler. Gör reglerna inställbara utan omkompilering.

Skapa ett `RuleSet` som läses in från en textfil vid start och skickas in där det behövs. Minst dessa inställningar:

- Drar givaren på mjuk 17, eller stannar hen?
- Hur många lekar spelas det med?
- Vid hur många kvarvarande kort blandas det om?
- Vinner naturlig tjugoett över en 21 byggd av tre kort?

Saknas filen ska programmet starta med rimliga standardvärden och säga till — det ska inte krascha.

Det svåra är inte filformatet. Det svåra är att hitta **varenda** hårdkodad regel i koden, och de ligger utspridda i andras moduler. Räkna med att läsa all gemensam kod och prata med alla i gruppen.

4. Satsningar och marker
Kräver ändringar i gemensam kod — `Player` och `Round`. Vänta tills grunden står.

Grundversionen räknar bara vunna och förlorade rundor. Lägg till en ekonomi.

- Varje spelare har ett antal marker och satsar innan given
- Vinst betalar 1:1 — insatsen tillbaka plus lika mycket till
- Naturlig tjugoett betalar 3:2
- Lika ger insatsen tillbaka
- En spelare som får slut på marker lämnar bordet
- Insatsen får inte vara större än spelarens marker, och inte noll

Markerna ska sparas mellan körningar, tillsammans med övriga spelaruppgifter.

Blir det tid över: lägg till **försäkring**. När givarens öppna kort är ett ess får spelaren satsa halva insatsen på att givaren har tjugoett. Har hen det betalar försäkringen 2:1, annars är den förlorad.

5. Sko med flera lekar och korträkning
Kräver ändringar i gemensam kod — `Deck`. Vänta tills grunden står.

Byt den enda kortleken mot en **sko**: flera lekar blandade till en. Lägg in ett skiljekort på slumpmässig plats i den bakre fjärdedelen; när det dras spelas rundan färdigt och därefter blandas skon om.

Lägg sedan på en korträknare som följer med i spelet. Hi-Lo-systemet ger varje kort ett värde:

Kort	Räknevärde
2, 3, 4, 5, 6	+1
7, 8, 9	0
10, knekt, dam, kung, ess	−1

Den **löpande räkningen** är summan av alla sedda kort. Den **sanna räkningen** är den löpande delad med antalet kvarvarande lekar. Visa båda, och en indikator för hur gynnsamt läget är.

Räknaren ska bara **observera**. Den får inte ändra någonting i spelet. Går det inte att bygga utan att röra reglerna har ni hittat ett problem värt att skriva om.

Överkurs, för den som blir klar tidigt
Dubbla: spelaren får ett enda kort till och måste sedan stanna. **Dela par:** två lika kort blir två händer som spelas var för sig. Delningen är svår — den ändrar hela rundans struktur, eftersom en spelare plötsligt har flera händer. Ge dig inte på den förrän ditt eget tillval är klart och mergat.

NBI/HANDELSAKADEMIN
GÖTEBORG · MALMÖ · HALMSTAD · VÄXJÖ · LULEÅ
+46 (0) 31-794 67 00
www.nbi-handelsakademin.se

Versionshantering

Ni arbetar som ett riktigt utvecklingsteam. Arbetssättet är en del av uppgiften och bedöms.

Branchstruktur

Branch	Roll
`main`	Den publicerade versionen. Rörs i princip inte under projektet. Hit mergas `dev` **en gång**, strax före inlämning.
`dev`	Huvudbranchen för utveckling. Ska vara **default branch** i repot.
`feature-...`	En per arbetsuppgift. Utgår alltid från `dev` och mergas tillbaka till `dev`.

Döp feature-brancherna så att det syns vad de är: `feature-hand-value`, `feature-dealer-rule`, `feature-card-graphics`.

Så arbetar du i en feature-branch

1. Utgå från senaste `dev`:
   `git switch dev && git pull && git switch -c feature-mitt-arbete`
2. Arbeta och commita löpande. Små commits med begripliga meddelanden.
3. **Innan du är klar: hämta in dev i din branch.**
   `git switch dev && git pull && git switch feature-mitt-arbete && git merge dev`
4. Lös eventuella konflikter **i din egen branch** och kontrollera att allt fortfarande fungerar.
5. Pusha och öppna en pull request mot `dev`.
6. En annan i gruppen granskar och godkänner. Sedan mergas den.

Steg 3 är det viktiga, och det som är lätt att hoppa över. Genom att ta in `dev` i din branch först löser du konflikterna där de hör hemma — i ditt eget arbete — i stället för att lämna dem till den som ska merga. Gör alla så håller sig `dev` hel hela tiden.

Skydd av brancherna

Sätt upp branch protection i repots inställningar:

Branch	Krav
`main`	Pull request krävs, **två** godkännande granskningar
`dev`	Pull request krävs, **en** godkännande granskning

Ingen får pusha direkt till `main` eller `dev`, inte heller den som äger repot.

NBI/HANDELSAKADEMIN
GÖTEBORG · MALMÖ · HALMSTAD · VÄXJÖ · LULEÅ
+46 (0) 31-794 67 00
www.nbi-handelsakademin.se

En i gruppen sätter upp repot. Utse en person och låt den göra hela uppsättningen ensam. Det är bara den som äger repot som kan lägga på skyddsreglerna, och sätter flera igång parallellt sitter ni plötsligt med tre repon att välja mellan.

Gör i den här ordningen, annars låser ni ut er själva:

1. Skapa repot och pusha den första koden till `main`
2. Skapa `dev` från `main`
3. Ställ in `dev` som default branch
4. Lägg på skyddsreglerna
5. Bjud in alla gruppmedlemmar som collaborators med skrivrättigheter
6. Övriga klonar repot och arbetar därefter utifrån `dev`

Repot ska vara publikt. På ett privat repo kräver GitHub ett betalt konto för att kunna sätta krav på granskning.

Om granskningar
Du kan inte godkänna din egen pull request — det tillåter inte GitHub, och det är poängen. En granskning ska vara en verklig genomläsning: förstår du vad koden gör, följer den gruppens struktur, finns det något som kommer att gå sönder?

Att granska andras kod är en del av bedömningen. Skriv kommentarer, ställ frågor, be om ändringar när det behövs.

Varför era commits spelar roll
Din individuella del bedöms bland annat på din branch och dina commits. Ett tillval som dyker upp i en enda stor commit kvällen före inlämning går inte att bedöma, och det går inte att se vad du själv har gjort. Arbeta löpande.

README och klassdiagram

Gruppen lämnar in en `README.md` i repots rot som innehåller:

[ ] Vad programmet gör och hur man startar det

[ ] En kort översikt över klasserna och hur de hänger ihop

[ ] Vem som gjort vad — vilket tillval var och en har byggt

[ ] Vilka regler som gäller, om ni avvikit från specifikationen på någon punkt

[ ] Kända brister

Dessutom ett **klassdiagram** över den färdiga koden. Det ska spegla hur programmet faktiskt ser ut vid inlämning, inte hur ni tänkte er det den första dagen. Lägg det som bild i repot och länka från README.

Individuell reflekterande rapport

Varje studerande skriver en egen rapport, cirka två till tre sidor, och lämnar in den på Learnpoint. Den ska innehålla:

Ditt tillval. Vad du byggde, vilka klasser du skapade och vilket ansvar var och en har.

Dina designval, motiverade. Inte vad du gjorde, utan varför. Vilka alternativ fanns, och varför valde du som du gjorde? Vad hade blivit sämre med det andra alternativet?

Ett problem du stötte på. Välj ett verkligt problem i din egen kod — en bugg, en struktur som inte höll, något du fick göra om. Beskriv vad som var fel, hur du kom på det, och hur du löste det. Föreslå dessutom en förbättring du skulle göra om du hade mer tid.

Samarbetet. Vad ändrades i den gemensamma koden på grund av ditt tillval? Vad lärde du dig av att granska någon annans kod?

Rapporten är ditt underlag för VG. Det som avgör är inte längden utan om du kan **motivera** — väga alternativ mot varandra och stå för ett val.

Redovisning

Hela gruppen redovisar tillsammans, cirka 20 minuter plus frågor.

- Demonstrera programmet. Spela en runda, visa minst två av tillvalen i användning.
- Gå igenom strukturen med klassdiagrammet.
- **Varje studerande redogör själv för sitt eget tillval och sina designval.** Det går inte att låta någon annan prata för dig.
- Räkna med frågor om kod du inte skrivit själv. Ni svarar för hela programmet.

NBI/HANDELSAKADEMIN
GÖTEBORG · MALMÖ · HALMSTAD · VÄXJÖ · LULEÅ
+46 (0) 31-794 67 00
www.nbi-handelsakademin.se

Bedömning

För Godkänt krävs

[ ] Programmet går att spela igenom enligt reglerna utan att krascha

[ ] Alla kraven på den gemensamma delen är uppfyllda

[ ] Regelklasserna innehåller ingen in- eller utmatning

[ ] Du har ett genomfört tillval med minst en egen klass, mergat till `dev` via en granskad pull request

[ ] Branchstrukturen och skyddsreglerna är som beskrivet ovan, och din branch har en begriplig commit-historik

[ ] Du har granskat minst en pull request från någon annan i gruppen

[ ] README och klassdiagram finns och stämmer

[ ] Din reflekterande rapport är inlämnad

[ ] Du kan redogöra för din egen kod med egna ord vid redovisningen

För Väl Godkänt krävs dessutom

[ ] Ditt tillval är tydligt avgränsat, med ett ansvar som går att beskriva i en mening

[ ] Du motiverar dina designval och kan väga dem mot alternativ du valde bort

[ ] Du identifierar och analyserar ett verkligt problem i din egen kod och föreslår en konkret förbättring

[ ] Koden håller: begripliga namn, hanterade fel, inga onödiga upprepningar

[ ] Dina granskningar av andras kod är innehållsrika, inte bara ett godkännande

VG sätts individuellt. Det går alldeles utmärkt att en grupp får olika betyg.

Checklista före inlämning

[ ] `dev` är mergad till `main` via en pull request med två godkännanden

[ ] Programmet startar och går att spela från `main`

[ ] README och klassdiagram är uppdaterade

[ ] Repots adress är inlämnad på Learnpoint

[ ] Din reflekterande rapport är inlämnad på Learnpoint

[ ] Alla i gruppen har commits på sin egen branch som visar eget arbete

Det viktigaste: ni ska kunna förklara all kod ni lämnar in, med egna ord. AI-verktyg och kursmaterial får användas som stöd, men kod ni inte kan redogöra för godkänns inte.

NBI/HANDELSAKADEMIN
GÖTEBORG · MALMÖ · HALMSTAD · VÄXJÖ · LULEÅ
+46 (0) 31-794 67 00
www.nbi-handelsakademin.se